Part 2 — Evolution and Refactoring

Team folder: A11 **Course:** 23CSE455 **Date:** 2026-10-11

Part 2 — Feature evolution and refactoring

Baseline and change sequence

The baseline is the supplied E-Learning-App source snapshot in `Part1/code/`. It contains course/user entities, REST endpoints, and the Angular authentication Observer, but no quiz model, quiz endpoints, scoring abstraction, or result notifications. The supplied ZIP has no `.git` history. The supplied patch files define the proposed sequential snapshots. No independently verifiable Git commit IDs or commit history were supplied; the previously recorded local hashes did not resolve to Git objects and are withdrawn. `changes/evolved_source.diff` is the complete baseline-to-final source comparison.

CH01 — Course quizzes and questions

Added `Quiz` and `Question` entities. A `Course` owns its quizzes; each quiz owns questions. New REST routes create a quiz for an existing course and list course quizzes. Input validation rejects absent titles, empty question sets, invalid prompt/answer text, and nonpositive/nonfinite points. API response DTOs omit correct answers. This is domain feature work, not an added pattern instance. The authentication Observer and its baseline flaw are unchanged at this stage.

CH02 — Selectable score policies

Introduced one Strategy collaboration named **Quiz Scoring Policy**. `QuizService` selects `StandardScoringStrategy` or `NegativeMarkingScoringStrategy` through the `ScoringStrategy` contract. Standard rules award points only for exact correct answers. Negative marking deducts half the question value for a wrong nonblank answer, ignores unanswered questions, and clamps to zero. Multiple concrete strategies are participants in one collaboration, counted as one instance.

CH03 — Persisted result notifications

Added a persisted `QuizResult`; after scoring, `QuizService.grade` saves the result and passes it to `QuizResultPublisher`. `StudentResultObserver` and `InstructorResultObserver` both receive the event through `QuizResultObserver`. The current adapters log availability. Email, push delivery, instructor-specific routing, and authentication/authorization enforcement are not implemented and must not be inferred from the observer names.

CH03 also fixes baseline auth-state drift. Login now calls `AuthService.setUser`; after a successful logout request, the service calls `clearUser`, which removes saved state and emits null. `HomeComponent` and

`NavbarComponent` are thereby kept in sync through the existing subject.

Pattern evolution and counts

`pattern_evolution.csv` retains the authentication Observer under the same instance name at each source version. Strategy appears after CH02. CH03 modifies the authentication Observer's update paths and adds a second, separate Observer collaboration for quiz results. The snapshots progress from one type/one instance to two types/two instances and finally two types/three instances. Counting a concrete participant class once per snapshot across all confirmed instances gives 3, 3, 6, and 9 respectively. Counting each unique mapped class/interface/method/field symbol once gives 4, 4, 8, and 13 under the role map in `pattern_evolution.csv`; shared `QuizService.grade` is counted once. Four named rejected candidates and one uncertain Factory candidate remain outside confirmed counts at each stage; two baseline-absent candidates become present after CH02/CH03. These are scoped candidate counts, not an exhaustive repository-wide catalog. The root `data/pattern_counts.csv` and `data/pattern_evolution.csv` mirror the collaboration records.

Verification and limits

Backend verification: `cd Part2/code/elearning-backend && bash ./mvnw test`. The final run passes 6/6 tests: two scoring test methods, two publisher tests, the Spring context test and an H2-backed integration test. Frontend verification: `cd Part2/code && npm ci && npm run build`; the production build succeeds with a bundle-size warning. Angular unit tests compile, but Karma cannot launch them because no Chrome binary is installed. Full interactive browser behavior is not covered. The runtime used here is Java 27 although Maven compiles the project to its declared Java 17 target; the local output records this.

The quiz feature is integrated into the Spring backend API and the auth-state correction is integrated into Angular.

Change traceability and measurements

Changed source paths in the supplied patch sequence

Change	Changed paths in its `diff.patch`
CH01	`elearning-backend/src/main/java/com/elearning/controller/QuizController.java`; `model/Course.java`; `model/Question.java`; `model/Quiz.java`; `repository/QuizRepository.java`; `service/QuizService.java` (paths after the `com/elearning/` prefix are relative to the backend Java source root).
CH02	`elearning-backend/src/main/java/com/elearning/controller/QuizController.java`; `service/QuizService.java`; `service/scoring/ScoringStrategy.java`; `StandardScoringStrategy.java`; `NegativeMarkingScoringStrategy.java`.

CH03	`README.md`; backend `QuizController.java`, `QuizResult.java`, `QuizResultRepository.java`, `QuizService.java`, `service/results/{QuizResultObserver,QuizResultPublisher,StudentResultObserver,InstructorResultObserver}.java`; backend tests `ElearningBackendApplicationTests.java`, `QuizServiceIntegrationTest.java`, `QuizResultPublisherTest.java`, `ScoringStrategyTest.java`, and the Mockito MockMaker config; frontend `src/app/components/login/login.component.ts`, `src/app/services/auth/auth.service.ts`, and `auth.service.spec.ts`. Full paths are recorded by the patch headers.

These paths describe the supplied proposed patch snapshots. They are not associated with verifiable Git commit IDs.

The ordered changes and changed source paths are listed in `changes/Change1/requirement.md` through `Change3/requirement.md` and their `diff.patch` files. Exact before/after Git commits cannot be reported because the baseline ZIP contains no Git metadata and no verified project commit history was supplied. The patches permit deterministic reconstruction of the staged trees but do not prove these are historical team commits.

A consistent source-size measure was computed from those staged trees. `evidence/measure_source_loc.py` reconstructs each stage from `Part1/code` plus the ordered patches; `evidence/source_loc_by_stage.csv` records file counts and nonblank physical lines. Scope is Angular `src/app/*/.ts` excluding spec files and backend `src/main/java/*/.java`; comments count, generated/dependency directories do not. Counts are Original 25 files/825 lines, CH01 30/914, CH02 33/959, CH03 39/1119. This is a transparent size proxy, not parser-derived executable LOC.

CH01 acceptance evidence: create/list course quizzes; reject invalid quiz/question input; omit answer keys from responses. CH02 acceptance evidence: standard score 7.5 and policy unit coverage for both implementations; integration grading exercises the selected policy. CH03 acceptance evidence: persist a result and notify each registered result observer; backend integration/publisher tests are in the saved output. No baseline-vs-after equivalence test was run for the auth refactor; the existing auth service test output is present, but browser execution is unavailable. Thus the refactoring's behavior preservation is only partially verified, not proven by a dedicated differential test. The source diffs are under `changes/` and test/build output under `evidence/`.

The actual original submission commit, a named human reviewer, and a source-level baseline test run were not supplied. Those fields remain unavailable rather than inferred. Run instructions for the evolved app are in `code/README.md`; the recorded final backend command is `cd Part2/code/elearning-backend && bash ./mvnw test`, and the frontend build command is `cd Part2/code && npm ci && npm run build`. Baseline build/tests were not run. This is not a finished production quiz system: authentication/role enforcement is only a request-level role check using the existing user data; student submissions and observer logs are not user-facing UI notifications; result delivery is synchronous and the observers are logging adapters. The app exposes correct answers only to the server and does not return them in quiz DTOs.